**Program Studi Teknik Informatika**
Institut Teknologi Sumatera

IF25-22014 PENGEMBANGAN APLIKASI WEB

Modul Belajar 7

# Kerangka Kerja React JS

Membangun antarmuka dari komponen React, dijalankan dengan Next.js

**Tim Pengajar**
Pengembangan Aplikasi Web. Tahun 2026

| Komponen | Keterangan |
|---|---|
| Pertemuan | 7 dari 16 |
| Capaian | Sub-CPMK 7.1 dan 7.2, turunan CPMK0502 |
| Waktu belajar | sekitar 3 jam belajar mandiri, ditambah 150 menit di kelas |
| Hands-on | `02 Hands-on/P7 - React JS - Hands-on/` |
| Yang perlu disiapkan | Node.js versi LTS, npm, VS Code, dan browser; koneksi internet untuk `npm install` |

## Peta belajar

Modul ini mengikuti bagian yang sama dengan slide. Setiap konsep inti ditulis dalam kotak gelap: itulah kalimat yang harus bisa kalian jelaskan ulang.

| Bagian | Judul | Yang dipelajari |
|---|---|---|
| 1 | Mengenal React dan Next.js | Menjelaskan React sebagai pustaka komponen dan Next.js sebagai kerangka kerjanya |
| 2 | JSX | Menulis tampilan dengan JSX dan memahami aturannya |
| 3 | Komponen, props, dan state | Membuat komponen yang menerima props dan menyimpan state |
| 4 | Siklus hidup komponen | Menjelaskan kapan komponen dipasang, diperbarui, dan dilepas |
| 5 | Hooks | Memakai hooks utama: useState, useEffect, useContext, useMemo, dan useRef |
| 6 | Pengambilan data | Mengambil data dari API dengan keadaan memuat dan error |
| 7 | Routing di Next.js | Membuat halaman, tautan, dan halaman dinamis dengan App Router |
| 8 | State management | Mengelola formulir dan berbagi state antarkomponen dengan React |

# Tentang modul ini

Modul ini menemani Pertemuan 7. Setiap kotak merah **Tebak dulu** meminta kalian menulis dugaan sebelum membaca kotak hijau **Jawaban** di bawahnya. Kerjakan contoh kode sambil membaca, jangan hanya dibaca.

### Setelah menyelesaikan modul ini, kalian bisa

| No | Kemampuan | Diukur di |
|---|---|---|
| 7.1 | Menjelaskan komponen, JSX, props, state, siklus hidup, dan hooks dalam membangun antarmuka | Exit ticket soal 1, hands-on, UTS, proyek |
| 7.2 | Membangun halaman React di Next.js dengan routing, pengambilan data, dan formulir terkendali | Exit ticket soal 2, hands-on, UTS, proyek |

Keduanya menurunkan CPMK0502: mahasiswa mampu menjelaskan konsep pemrograman untuk pengembangan perangkat lunak.

# Masalah pembuka

### Sepuluh kartu, sepuluh kali salin

Halaman KRS kalian dari P5 menampilkan setiap mata kuliah sebagai kartu. Kode kartunya disalin sepuluh kali. Saat desain kartu diubah, sepuluh tempat harus diedit.

Lalu total SKS di pojok halaman harus diperbarui manual setiap kali kartu dicentang, dan dua kali lupa.

**TEBAK DULU**

Bagaimana menulis kartu sekali saja, dan membuat total SKS selalu benar dengan sendirinya?

```
<div class="kartu">
  <h3>Pemweb</h3><p>3 SKS</p>
</div>
<div class="kartu">
  <h3>PAM</h3><p>3 SKS</p>
</div>
<!-- ... delapan lagi -->
<p>Total: <span id="t">6</span></p>
```

# Bagian 1: Mengenal React dan Next.js

**TUJUAN: MENJELASKAN REACT SEBAGAI PUSTAKA KOMPONEN DAN NEXT.JS SEBAGAI KERANGKA KERJANYA**

## Pustaka dan kerangka kerjanya

> **React dan Next.js**
> **React** adalah pustaka JavaScript untuk membangun antarmuka dari komponen. **Next.js** adalah kerangka kerja di atas React yang menyediakan routing, rendering di server, dan alat build.

React sendiri hanya mengurus tampilan. Hal lain seperti halaman, URL, dan pengambilan data disediakan Next.js. Proyek mata kuliah ini memakai Next.js sebagai frontend yang memanggil API FastAPI.

```
npx create-next-app@latest krs-app
cd krs-app
npm run dev
# buka http://localhost:3000
```

## Struktur proyek Next.js

| Berkas atau folder | Isinya |
|---|---|
| `app/page.js` | halaman `/`; inilah jawaban misi 1 |
| `app/layout.js` | kerangka yang membungkus semua halaman: html, body, menu |
| `app/krs/page.js` | halaman `/krs`: nama folder menjadi path |
| `components/` | komponen yang dipakai ulang, misalnya kartu |
| `*.module.css` | CSS khusus satu komponen; nama kelas tidak bertabrakan |
| `public/` | berkas statis: gambar, JSON contoh |
| `package.json` | daftar pustaka dan perintah `dev`, `build`, `start` |

## Ikuti bersama: proyek React pertama

Kerjakan sambil membaca, satu langkah demi satu langkah.

| No | Langkah | Perintah atau kode |
|---|---|---|
| 1 | Buka terminal di folder `starter/`, pasang pustaka. | `npm install` |
| 2 | Jalankan server pengembangan. | `npm run dev` |
| 3 | Buka alamatnya di browser. | `http://localhost:3000` |
| 4 | Buka `app/page.js`, ganti teks judul dengan nama kalian, lalu simpan. | |
| 5 | Lihat browser tanpa memuat ulang. | |

> **YANG HARUS TERLIHAT**
> Terminal menulis *Ready* dan alamat `localhost:3000`. Judul di browser berubah sendiri sesaat setelah berkas disimpan.

**Kalau berbeda**

*npm tidak dikenal*: pasang Node.js, buka terminal baru. *Port 3000 is in use*: buka port lain yang ditulis terminal. Judul tetap: simpan `app/page.js`.

**Inti Bagian 1**

React membangun antarmuka dari komponen; Next.js menambah routing, rendering server, dan build.

# Bagian 2: JSX

**TUJUAN: MENULIS TAMPILAN DENGAN JSX DAN MEMAHAMI ATURANNYA**

## Aturan JSX

```
export default function Profil() {
  const nama = "Rani";
  const sks = 20;
  return (
    <section className="kartu">
      <h2>{nama}</h2>
      <p>{sks ≥ 24 ? "SKS penuh" : `${sks} SKS`}</p>
      <img src="/foto.jpg" alt={`Foto ${nama}`} />
      <label htmlFor="cari">Cari</label>
    </section>
  );
}
```

JSX terlihat seperti HTML, tetapi sebenarnya JavaScript yang diubah menjadi pemanggilan fungsi.

`{ }` menyisipkan ekspresi JavaScript: variabel, operator ternary, template literal. Bukan pernyataan seperti `if`.

`class` menjadi `className`, `for` menjadi `htmlFor`, karena keduanya kata kunci JavaScript.

Elemen kosong wajib ditutup: `<img />`. Kembalikan satu induk, atau bungkus dengan `<> ... </>`.

## Gaya dengan CSS Modules

**components/kartu.module.css**

```
.kartu {
  border: 2px solid var(--utama);
  border-radius: 8px;
  padding: 12px 16px;
}
.kartu:hover { background: #F6EDCF; }
```

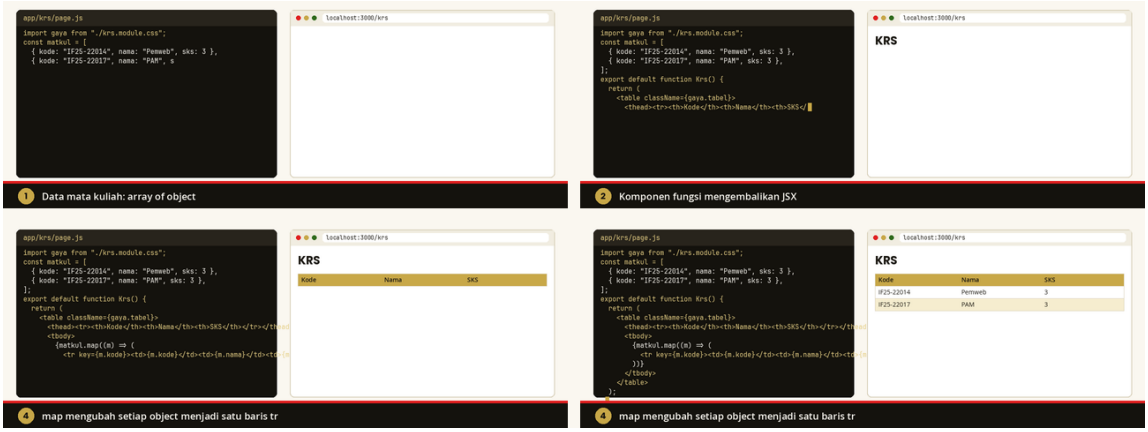
**components/KartuMatkul.js**

```
import gaya from "./kartu.module.css";

export default function KartuMatkul({ nama, sks }) {
  return <article className={gaya.kartu}>
    <h3>{nama}</h3><p>{sks} SKS</p>
  </article>;
}
```

CSS dari P3 dan P4 dipakai apa adanya. Nama kelas diubah otomatis menjadi unik per komponen, jadi `.kartu` di dua komponen tidak saling menimpa. Gaya umum seperti variabel `--utama` ditulis di `app/globals.css`.

### Dari data ke tabel




Satu komponen, satu array, satu map: tabel terbangun tanpa pustaka tambahan. Versi bergeraknya ada di slide PowerPoint.

**Inti Bagian 2**

JSX mirip HTML di JavaScript: className, kurung kurawal untuk ekspresi, satu induk.

# Bagian 3: Komponen, props, dan state

**TUJUAN: MEMBUAT KOMPONEN YANG MENERIMA PROPS DAN MENYIMPAN STATE**

### Blok pembangun antarmuka

**Komponen**

Fungsi JavaScript berawalan huruf besar yang menerima **props** dan mengembalikan JSX; satu komponen bisa dipakai berkali-kali.

Inilah jawaban masalah minggu ini: kartu ditulis sekali sebagai komponen, lalu dipakai sepuluh kali dengan data berbeda lewat `map`. Setiap item daftar butuh `key` unik supaya React tahu mana yang berubah.

```
function KartuMatkul({ nama, sks }) {
  return <div className="kartu">
    <h3>{nama}</h3><p>{sks} SKS</p>
  </div>;
}
// di halaman:
{matkul.map((m) => (
  <KartuMatkul key={m.kode} nama={m.nama} sks={m.sks} />
))}
```

### State: data yang berubah

```
"use client";
import { useState } from "react";

export default function Krs({ matkul }) {
  const [diambil, setDiambil] = useState([]);
  const total = diambil.reduce((a, m) => a + m.sks, 0);

  const ambil = (m) => setDiambil([...diambil, m]);

  return (<>
    {matkul.map((m) => (
      <button key={m.kode} onClick={() => ambil(m)}>{m.nama}</button>
    ))}
    <p>Total: {total} SKS</p>
  </>);
}
```

Bagian kedua masalah minggu ini: total yang selalu benar.

`useState([])` memberi nilai sekarang dan fungsi pengubahnya. `"use client"` wajib karena state hidup di browser.

`total` tidak disimpan; ia dihitung ulang dari state setiap kali tampil, jadi tidak mungkin lupa diperbarui.

Ubah state dengan fungsi pengubah dan salinan baru (spread), jangan `diambil.push(m)`; React hanya bereaksi pada nilai baru.

`onClick={() => ambil(m)}`: kirim fungsi, bukan memanggilnya. Kesalahan yang sama dengan P5.

### Props atau state?

**TEBAK DULU**

Komponen `KartuMatkul` menampilkan nama mata kuliah dari induknya dan tanda centang yang berubah saat kartunya diklik. Mana yang props dan mana yang state? (A) keduanya props; (B) nama props, centang state; (C) nama state, centang props; (D) keduanya state

**JAWABAN: B**

Nama datang dari luar dan tidak diubah kartu: props. Centang berubah karena tindakan di kartu itu sendiri: state.

**Inti Bagian 3**

Komponen fungsi pengembali JSX; props dari induk dibaca saja, state milik komponen.

## Bagian 4: Siklus hidup komponen

**TUJUAN: MENJELASKAN KAPAN KOMPONEN DIPASANG, DIPERBARUI, DAN DILEPAS**

### Tiga tahap kehidupan

Komponen fungsi menjalani tiga tahap. Kelas komponen lama punya method untuk tiap tahap; sekarang semuanya lewat `useEffect`.

| 1 Mounting | 2 Updating | 3 Unmounting |
|---|---|---|
| Pertama kali tampil. Tempat yang tepat untuk mengambil data awal. | Tampil ulang setiap props atau state berubah. Fungsi komponen dijalankan lagi dari atas. | Hilang dari halaman, misalnya pindah halaman. Bersihkan timer atau langganan di sini. |

Karena fungsi komponen dijalankan ulang setiap update, jangan memanggil `fetch` langsung di badan fungsi; pakai `useEffect`.

**Inti Bagian 4**

Dipasang, diperbarui saat props atau state berubah, lalu dilepas; efek lewat useEffect.

# Bagian 5: Hooks

**TUJUAN: MEMAKAI HOOKS UTAMA: USESTATE, USEEFFECT, USECONTEXT, USEMEMO, DAN USEREF**

## useEffect

```
useEffect(() => {
  const id = setInterval(() => setDetik((d) => d + 1), 1000);
  return () => clearInterval(id); // saat dilepas
}, []); // [] = sekali saat dipasang

useEffect(() => {
  document.title = `KRS (${total} SKS)`;
}, [total]); // setiap total berubah
```

`useEffect(fungsi, daftar)` menjalankan efek sesudah komponen tampil.

Daftar kosong `[]`: hanya sekali saat dipasang.

Fungsi yang dikembalikan dijalankan saat dilepas: tempat membersihkan timer.

Daftar `[total]`: jalan lagi setiap `total` berubah. Lupa daftar sama sekali berarti jalan setiap tampil, sumber perulangan tanpa akhir bila efeknya mengubah state.

## Hooks lain yang perlu dikenal

| Hook | Gunanya | Contoh |
|---|---|---|
| `useContext` | berbagi data ke banyak komponen tanpa diteruskan lewat props berlapis | tema gelap, data pengguna yang login |
| `useMemo` | mengingat hasil hitungan berat sampai masukannya berubah | menyaring 2000 baris nilai |
| `useRef` | menyimpan nilai atau elemen DOM tanpa menggambar ulang | memfokuskan kotak isian |
| `useReducer` | state rumit dengan banyak aksi | keranjang KRS dengan tambah, hapus, kosongkan |

**Inti Bagian 5**

useState menyimpan, useEffect menjalankan efek, useContext berbagi data.

# Bagian 6: Pengambilan data

**TUJUAN: MENGAMBIL DATA DARI API DENGAN KEADAAN MEMUAT DAN ERROR**

### fetch di komponen klien

```
"use client";
export default function DaftarMatkul() {
  const [data, setData] = useState([]);
  const [memuat, setMemuat] = useState(true);
  const [galat, setGalat] = useState(null);
  useEffect(() => {
    fetch("http://localhost:8000/matkul")
      .then((r) => r.ok ? r.json() : Promise.reject(r.status))
      .then(setData).catch(setGalat)
      .finally(() => setMemuat(false));
  }, []);
  if (memuat) return <p>Memuat mata kuliah...</p>;
  if (galat) return <p role="alert">Gagal memuat. Coba lagi.</p>;
  return <ul>{data.map((m) => <li key={m.kode}>{m.nama}</li>)}</ul>;
}
```

Pola yang akan dipakai di proyek: halaman Next.js memanggil API FastAPI di port 8000.

Tiga state: data, sedang memuat, dan error. Inilah keadaan antarmuka dari P1 dan P6.

Ambil data sekali saat dipasang: `useEffect` dengan `[]`.

Tampilkan sesuai keadaan: teks memuat, pesan gagal dengan `role="alert"` agar dibacakan pembaca layar, lalu daftar saat berhasil.

Komponen server di Next.js bisa langsung `await fetch(...)` tanpa state. Pustaka seperti SWR atau TanStack Query menambah cache dan muat ulang otomatis.

**Inti Bagian 6**

fetch di useEffect dengan state data, memuat, dan galat; komponen server boleh await.

# Bagian 7: Routing di Next.js

**TUJUAN: MEMBUAT HALAMAN, TAUTAN, DAN HALAMAN DINAMIS DENGAN APP ROUTER**

### Folder menjadi URL

| Berkas | URL |
|---|---|
| `app/page.js` | `/` |
| `app/tentang/page.js` | `/tentang`: jawaban misi 2 |
| `app/matkul/page.js` | `/matkul` |
| `app/matkul/[kode]/page.js` | `/matkul/IF25-22014`, `/matkul/IF25-22017`, dan seterusnya |
| `app/layout.js` | membungkus semua halaman di bawahnya |

### Link dan halaman dinamis

app/matkul/page.js

```
import Link from "next/link";

<Link href={`/matkul/${m.kode}`}>
  {m.nama}
</Link>
```

app/matkul/[kode]/page.js

```
"use client";
import { useParams, useRouter }
  from "next/navigation";

export default function Detail() {
  const { kode } = useParams();
  const router = useRouter();
  return <button onClick={() =>
    router.push("/matkul")}>{kode}</button>;
}
```

`Link` berpindah tanpa memuat ulang; `[kode]` menangkap bagian URL; `useParams` membacanya.

**Inti Bagian 7**

Folder menjadi path, page.js halamannya, [kode] halaman dinamis, Link berpindah.

# Bagian 8: State management

**TUJUAN: MENGELOLA FORMULIR DAN BERBAGI STATE ANTARKOMPONEN DENGAN REACT**

### Formulir terkendali

```
export default function Tambah({ onTambah }) {
  const [nama, setNama] = useState("");
  const [galat, setGalat] = useState("");
  function kirim(e) {
    e.preventDefault();
    if (nama.trim().length < 3) return setGalat(
      "Nama minimal 3 huruf.");
    onTambah({ nama }); setNama("");
  }
  return <form onSubmit={kirim}>
    <label htmlFor="nama">Nama</label> <input id="nama"
      value={nama} onChange={(e) => {
        setNama(e.target.value); setGalat(""); }} />
    {galat && <p role="alert">{galat}</p>}
    <button type="submit">Tambah mata kuliah</button>
  </form>;
}
```

Formulir P4 dan P5, sekarang versi React.

**Terkendali**: `value` dari state, `onChange` memperbarui state. React selalu tahu isi kotaknya.

Validasi saat kirim; pesan galat menjelaskan cara memperbaiki dan hilang saat pengguna mulai mengetik (pedoman P4).

Tombol menyebut aksinya; pesan berhasil dalam bentuk lampau yang spesifik. `role` membuat pesan dibacakan pembaca layar.

Pola yang sama dipakai untuk menampilkan pesan 422 dari FastAPI di P11.

## Context dan useReducer

```
"use client"; // app/KrsProvider.js
const KrsContext = createContext(null);
function reducer(state, aksi) { switch (aksi.jenis) {
    case "ambil": return [...state, aksi.matkul];
    case "lepas":
      return state.filter((m) => m.kode !== aksi.kode);
    default: return state;
}}
export function KrsProvider({ children }) {
  const [diambil, kirim] = useReducer(reducer, []);
  return <KrsContext.Provider value={{ diambil, kirim }}>
    {children}
  </KrsContext.Provider>;
}
export const useKrs = () => useContext(KrsContext);
```

Masalah: total SKS di menu atas dan daftar di halaman butuh state yang sama, padahal letaknya berjauhan.

`createContext` dan `Provider`: state disediakan sekali di `layout.js`, dibaca di mana pun dengan `useKrs()`.

`useReducer`: semua perubahan state lewat satu fungsi dengan aksi bernama, lebih mudah dilacak daripada banyak `set...`.

Untuk aplikasi besar, pustaka seperti Redux atau Zustand memakai gagasan yang sama. State management ini dari slide lama.

## Tiga baris yang salah

| Yang ditulis | Mengapa salah, dan perbaikannya |
|---|---|
| `useState` di `page.js` tanpa `"use client"` | Komponen di folder app adalah komponen server secara bawaan; hooks tidak tersedia. Tambahkan `"use client"` di baris pertama. |
| `{matkul.map((m) => <Kartu nama={m.nama} />)}` | Tanpa `key`; React memberi peringatan dan bisa salah memperbarui baris. Tambahkan `key={m.kode}`. |
| `diambil.push(m); setDiambil(diambil);` | Array yang sama diubah langsung, React tidak melihat perubahan. Pakai `setDiambil([...diambil, m])`. |

**Pola kesalahannya**

Dua yang terakhir tidak memunculkan error merah, hanya tampilan yang tidak berubah atau peringatan di Console.

## Giliran kalian

**SETENGAH JADI: PENGHITUNG SKS**

Lengkapi: `const [total, _____] = useState(0);` dan tombol `<button onClick={_____}>Tambah 3 SKS</button>`.

**MANDIRI: HALAMAN DETAIL**

Tulis `app/dosen/[nip]/page.js` yang menampilkan NIP dari URL di dalam `<article>` bergaya CSS Modules, dan tombol *Kembali ke daftar dosen*.

**JAWABAN**

Setengah jadi: `setTotal`, `() => setTotal(total + 3)`. Mandiri: ”use client”, useParams untuk nip, useRouter untuk kembali, import gaya dari berkas module.css.

**Inti Bagian 8**

Formulir terkendali menyimpan isian di state; Context dan useReducer membagikan state ke banyak komponen tanpa props berlapis.

# Latihan mandiri

Kerjakan lembar dan berkas awal di folder `02 Hands-on/P7 - React JS - Hands-on/`.

| Tingkat | Waktu | Yang dilatih |
|---|---|---|
| Tingkat 1, Dasar | 10′ | komponen dengan props, CSS Modules, dan `map` dengan `key` |
| Tingkat 2, Menengah | 15′ | state total SKS dan pengambilan data dengan keadaan memuat dan gagal |
| Tingkat 3, Tantangan | 10′ | halaman dinamis, Link, tabel, dan formulir terkendali |
| Pengayaan | sisa | Context untuk total SKS di menu atas |

## Kesalahan yang sering muncul

| Yang ditulis | Masalahnya | Obatnya |
|---|---|---|
| hooks tanpa `"use client"` | komponen server tidak punya state | tambahkan direktif di baris pertama |
| `map` tanpa `key` | React salah melacak item | `key` unik dari data |
| `state.push(x)` | mengubah langsung, tidak tampil ulang | salinan baru dengan spread |
| `fetch` di badan komponen | jalan setiap render, bisa tak berujung | di dalam `useEffect` |
| `class=` di JSX | kata kunci JavaScript | `className` |
| input tanpa `value` dan `onChange` | isian dan state tidak sinkron | formulir terkendali |

# Rangkuman

| No | Bagian | Inti |
|---|---|---|
| 1 | Mengenal React dan Next.js | React membangun antarmuka dari komponen; Next.js menambah routing, rendering server, dan build. |
| 2 | JSX | JSX mirip HTML di JavaScript: className, kurung kurawal untuk ekspresi, satu induk. |
| 3 | Komponen, props, dan state | Komponen fungsi pengembali JSX; props dari induk dibaca saja, state milik komponen. |
| 4 | Siklus hidup komponen | Dipasang, diperbarui saat props atau state berubah, lalu dilepas; efek lewat useEffect. |
| 5 | Hooks | useState menyimpan, useEffect menjalankan efek, useContext berbagi data. |
| 6 | Pengambilan data | fetch di useEffect dengan state data, memuat, dan galat; komponen server boleh await. |
| 7 | Routing di Next.js | Folder menjadi path, page.js halamannya, [kode] halaman dinamis, Link berpindah. |
| 8 | State management | Formulir terkendali menyimpan isian di state; Context dan useReducer membagikan state ke banyak komponen tanpa props berlapis. |

# Cek pemahaman

Jawab tanpa membuka modul atau slide. Mengingat kembali tanpa melihat lebih menguatkan ingatan daripada membaca ulang.

| No | Soal |
|---|---|
| 1 | Apa beda React dan Next.js? |
| 2 | Mengapa `for` menjadi `htmlFor` di JSX? |
| 3 | Apa beda props dan state? |
| 4 | Kapan `useEffect(f, [])` dijalankan? |
| 5 | Berkas apa yang menghasilkan halaman `/matkul/IF25-22014`? |
| 6 | Apa itu formulir terkendali? |

**KUNCI CEK PEMAHAMAN**

(1) React pustaka komponen; Next.js kerangka kerja React dengan routing, rendering server, dan build. (2) for adalah kata kunci JavaScript. (3) Props dari induk dan hanya dibaca; state milik komponen dan berubah. (4) Sekali, sesudah komponen pertama kali tampil. (5) app/matkul/[kode]/page.js. (6) Input yang nilainya diambil dari state dan diubah lewat onChange, sehingga React selalu tahu isinya.

# Bacaan lanjut

React, *Learn React*, termasuk *Managing State* dan *Passing Data Deeply with Context* (`react.dev/learn`). Next.js, *Getting Started*, *Routing*, dan *CSS Modules* (`nextjs.org/docs`). Pustaka lama: Algifari dkk., ITERA Press, 2024, bab React JS, untuk konsep komponen dan state.